

# NLP Misfortune Classification Project - Model Documentation

## Final Model and Training Methodology

### Executive Summary

This document describes the development and evaluation of multi-label classification models for the HRAF (Human Relations Area Files) misfortune dataset. The project achieved a **flat model F1 micro score of 0.722** using quality-stratified training data.

Key modifications include:

- **Quality-based data stratification** using semantic embeddings and reranking scores
- **Weighted focal loss** to address severe class imbalance

After experimenting with hierarchical training configurations, the flat model architecture proved most effective for this task.

## 1. Best Performing Model (training\_20251014\_124910(best))

### 1.1 Data Loading and Preparation

**Source Data:** \_Altogether\_Dataset\_RACoded\_Combined.xlsx (11.005 unique passages, 10388 after deduplication)

#### Deduplication Strategy:

- Removed exact duplicate passages
- Manually identified near-duplicates
- Prioritized Run 3 passages when conflicts existed
- Generated stable 16-character content-hash-based IDs (MD5 hash) for each passage

**Label Structure:** The flat model treats all 12 labels independently without hierarchical constraints:

| Category   | Labels                                                                                          |
|------------|-------------------------------------------------------------------------------------------------|
| EVENT (3)  | Illness, Accident, Other                                                                        |
| CAUSE (4)  | Material_Physical, Spirits_Gods, Witchcraft_Sorcery, Rule_Violation_Taboo                       |
| ACTION (5) | Physical_Material, Technical_Specialist, Divination, Shaman_Medium_Healer, Priest_High_Religion |

### 1.2 Golden Dataset Creation

**Objective:** Identify high-quality training passages using semantic analysis rather than relying solely on inter-rater agreement (Kappa: 0.02-0.67).

**Process:**

1. **Embedding Generation** (VoyageAI voyage-3-large)
  - Generated 1024-dimensional embeddings for all 10,388 passages
  - Stored in Pinecone vector database (namespace: `raw__altogether_dataset_racoded_combined_20251014`)
  - Batch size: 32 passages
2. **Consistency Score Calculation**
  - For each passage, retrieved k=20 most similar passages
  - Calculated per-label consistency using frequency-weighted agreement:
    - **Very rare labels** (<5%): Required high similarity to labeled neighbors
    - **Rare labels** (5-15%): Hybrid of agreement + similarity
    - **Common labels** (>15%): Traditional neighbor agreement
  - Formula:  $\text{consistency\_avg} = \frac{\sum(\text{consistency\_label} \times \text{weight\_label})}{\sum(\text{weights})}$
  - Weight =  $1 / (\text{label\_frequency} + 0.01)$
3. **Rerank Score Calculation** (VoyageAI rerank-2.5)
  - Evaluated semantic relevance of each passage to its assigned labels
  - Label-specific queries (e.g., "Illness": "Disease, sickness, illness, or mental/physical health problems")
  - Scored each passage-label pair for semantic coherence
  - Relevance scores range [0, 1], normalized across labels
4. **Composite Quality Score**
  - $\text{composite} = 0.5 \times \text{consistency\_avg} + 0.5 \times \text{rerank\_avg}$
  - Higher scores indicate passages where:
    - Similar passages have consistent labels (consistency)
    - Passage text semantically matches its labels (rerank)
5. **Quality Tier Assignment** - Using the 5,000 highest-scoring passages and mixed inference set of 2,086 (total 7086 passages) :

| Label Distribution in Training Set (5,000 passages): | Size  | Quality Thresholds                                | Purpose                                        |
|------------------------------------------------------|-------|---------------------------------------------------|------------------------------------------------|
| <b>Tier 1</b>                                        | 2,000 | Consistency $\geq 0.731$ ,<br>Rerank $\geq 0.511$ | Elite training data (top ~20% of dataset)      |
| <b>Tier 2</b>                                        | 3,000 | Consistency $\geq 0.571$ ,<br>Rerank $\geq 0.407$ | Expansion training data (next ~30% of dataset) |
| <b>Inference</b>                                     | 2,086 | Below Tier 2 thresholds                           | Test/evaluation set (~20%)                     |

### 1.3 Model Training Configuration

**Architecture:** RoBERTa-base (12 layers, 768 hidden dimensions)

**Training Strategy:** Combined Tier 1 + Tier 2 (5,000 passages total)

**Key Hyperparameters:**

Base Model: roberta-base

Labels: 12 (flat, no hierarchy)

Hidden Layers (Classifier): 3

Hidden Size (Classifier): 768

Dropout: 0.15

Attention Dropout: 0.10

Loss Configuration:

- Focal Loss: Enabled ( $\gamma = 4.0$ )
- Weighted Loss: Enabled (class frequency-based)
- Combined: Weighted Focal Loss

Training Parameters:

- Epochs: 13
- Batch Size: 12
- Learning Rate:  $2e-5$
- Warmup Steps: 500
- Weight Decay: 0.02
- Max Sequence Length: 512 tokens

Data Split:

- Train: 3,600 passages (72%)
- Validation: 400 passages (8%)
- Test: 1,000 passages (20%)
- Random Seed: 42

**Critical Design Choices:**

**1. Weighted Focal Loss ( $\gamma=4.0$ ):** Addresses severe class imbalance

- Priest\_High\_Religion: 2.7%  $\rightarrow$  weight = 100 $\times$  (capped)
- Illness: 67.6%  $\rightarrow$  weight = 0.48 $\times$
- Focuses learning on hard-to-classify examples

**2. Quality-Stratified Data:** Uses semantically verified passages rather than all available data

- Training on 5,000 high-quality passages vs. 10,367 total
- Reduces label noise from inconsistent annotations

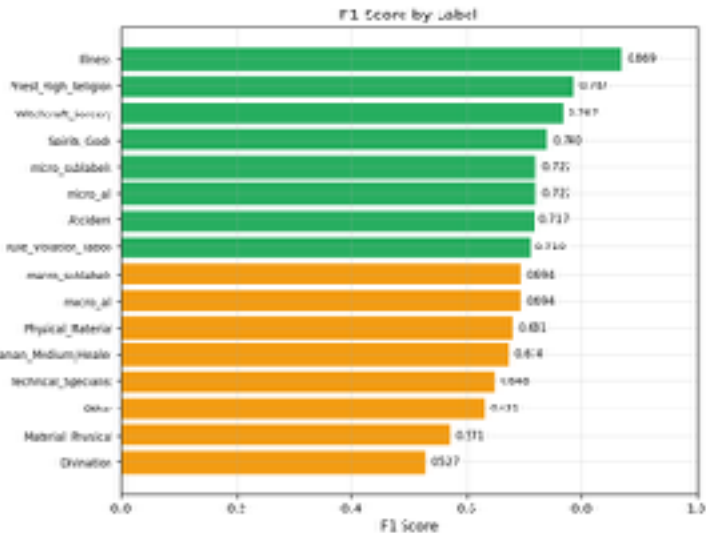
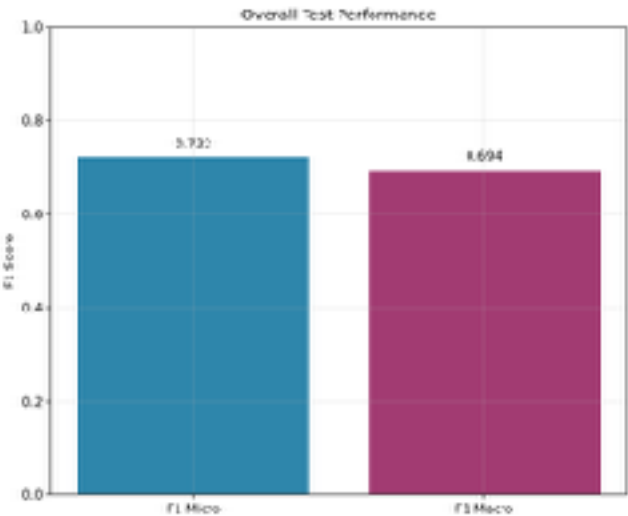
**3. Flat Architecture:** All labels predicted independently

- Simpler than hierarchical (no gating, no main categories)
- More stable training
- Better performance on this dataset

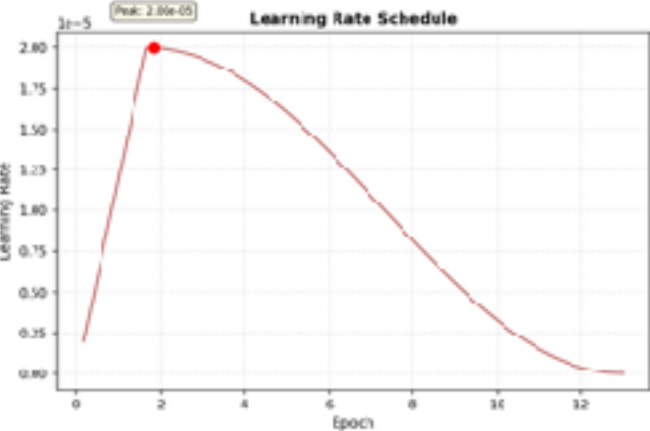
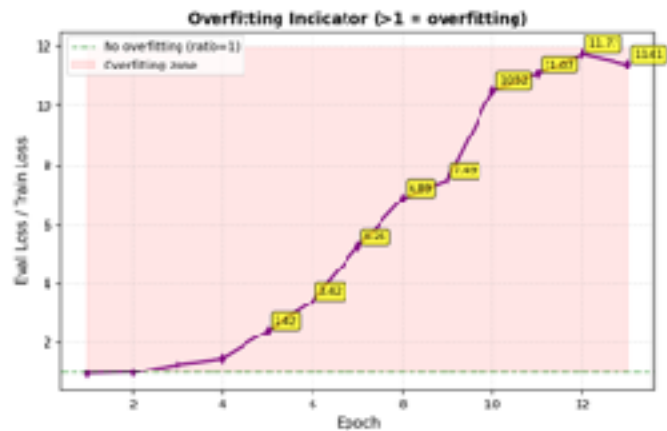
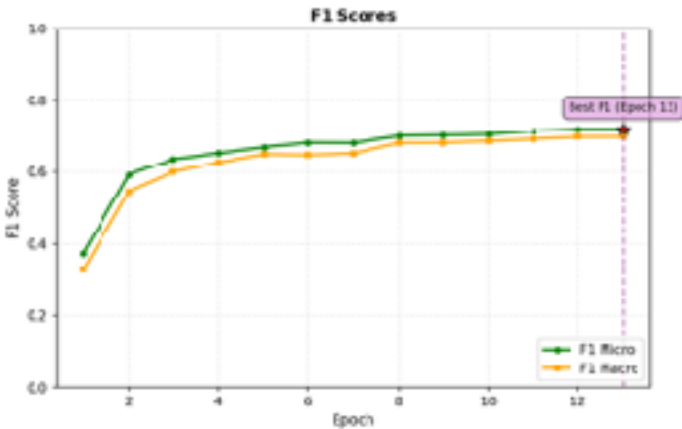
**1.4 Training Results**

Test Set Performance (1,000 passages):

|                   |       |
|-------------------|-------|
| F1 Micro          | 0.722 |
| F1 Macro          | 0.694 |
| Precision (Micro) | 0.718 |
| Recall (Micro)    | 0.726 |



Training and Validation Loss



## 2. Reranker System and Quality Scoring

### 3.1 What is Reranking?

**Reranking** evaluates the semantic relevance between a passage's text content and its assigned labels. Unlike traditional classification (which learns from label patterns), reranking uses pre-trained language understanding to assess whether a passage's text actually describes what the label claims.

#### Example:

Passage: "The healer called upon the spirits to cure the child's fever..."

Label: "Shaman\_Medium\_Healer"

Rerank Score: 0.87 ← High relevance

Passage: "He washed the wound with clean water and bandaged it..."

Label: "Shaman\_Medium\_Healer"

Rerank Score: 0.23 ← Low relevance (misabeled, should be Physical\_Material)

### 3.2 Score Definitions

#### Consistency Score (0-1):

- Measures agreement between similar passages
- Higher = neighbors have matching labels
- Weighted by label rarity to avoid common-label bias

#### Rerank Score (0-1):

- Measures semantic match between text and labels
- Uses VoyageAI rerank-2.5 API
- Label-specific semantic queries (see Appendix A)

#### Composite Score (0-1):

- $\text{composite} = 0.5 \times \text{consistency} + 0.5 \times \text{rerank}$
- Used for final tier assignment
- Balances neighbor agreement with semantic coherence

### 3.3 Why This Matters

Traditional inter-rater agreement (Kappa: 0.02-0.67) was insufficient for quality control. The reranker system provides an alternative quality signal based on:

1. **Semantic coherence:** Does the text match what the label means?
2. **Label consistency:** Do similar passages receive similar labels?
3. **Objective measurement:** Not dependent on any single annotator's interpretation

This allowed us to identify the 5,000 most reliable passages for training, improving model performance despite reducing training set size.

### 3. Golden Dataset Application

#### 3.1 Installation and Setup

##### Prerequisites:

```
bash
# Install dependencies
pip install -r requirements.txt
```

```
# Create .env file with API keys
echo "VOYAGE_API_KEY=your_voyage_key" > .env
echo "PINECONE_API_KEY=your_pinecone_key" >> .env
```

##### Launch Application:

```
bash
streamlit run app_golden_dataset.py
```

The application will open in your browser at <http://localhost:8501>

#### 3.2 Navigation Overview

The application has four main pages with navigation in the sidebar:

| Page            | Purpose                                      |
|-----------------|----------------------------------------------|
| <b>Data</b>     | Load, clean, embed, score, and tier datasets |
| <b>Analysis</b> | Explore data quality and distributions       |
| <b>Models</b>   | Train, evaluate, and compare models          |
| <b>Discover</b> | Semantic search and hypothesis testing       |

#### 3.3 Step-by-Step: Creating Golden Dataset

##### Step 1: Load Data

1. Navigate to **Data** page
2. Click **Create New** tab
3. Upload Excel file (\_Altogether\_Dataset\_RACoded\_Combined.xlsx)
4. Select header row (row 1)
5. Select passage column (e.g., "Passage")
6. Check all metadata to be included with data

7. Check training labels
8. Click Confirm Configuration
9. Enter object name (e.g., `raw_hraf_2024`)
10. Click **Create RAW Data Object**

#### **Step 2: Clean Data (Optional)**

1. Click **Analyze Data Quality**
2. Review detected issues (duplicates, missing, etc.)
3. Select cleaning steps to apply
4. Enter cleaned object name
5. Click **Apply Cleaning OR Skip Cleaning**

#### **Step 3: Generate Embeddings**

1. Ensure Voyage AI API key is set
2. Set batch size (default: 32)
3. Enter embedded object name
4. Click **Generate Embeddings**
5. Wait for completion (~10-15 minutes for 10,000 passages)

#### **Step 4: Calculate Quality Scores**

1. Set `k_similar` (default: 20)
2. Enter scored object name
3. Click **Calculate Scores**
4. Reranking will evaluate each label (~15-20 minutes)

#### **Step 5: Create Training Tiers**

1. Review score distributions in the explorer
2. Choose configuration method:
  - **Smart Presets:** Recommended (uses data-aware defaults)
  - **Manual Thresholds:** Custom control with live preview
  - **Skip Quality Filtering:** Random stratified split
3. Enable label targeting for rare labels (recommended)
4. Click **Create Tiers**

**Result:** Three datasets saved:

- `tier1.xlsx` - Elite training data
- `tier2.xlsx` - Expansion training data
- `inference.xlsx` - Test/evaluation data

### **3.4 Step-by-Step: Training a Model**

### Step 1: Navigate to Models Page

1. Click **Models** in sidebar
2. Select **Train New Model** tab

### Step 2: Select Training Data

1. Under "Dataset Selection", choose:
  - **Current Dataset** (if already loaded)
  - **Browse Tiered Data** (to load saved tiers)
  - **Browse Experiments** (to reuse previous configs)
2. If browsing tiers, select desired tier combination:
  - Tier 1 Only (conservative, highest quality)
  - Tier 1 + Tier 2 Combined (recommended)
  - Curriculum Learning (start tier 1, expand to combined)
  -

### Step 3: Configure Model Architecture

1. Select base model (roberta-base recommended)
2. Set classifier hidden layers (3 recommended)
3. Set hidden size (768 for roberta-base)
4. Set dropout rates (0.15 and 0.10 recommended)
5. For hierarchical: Enable and configure hierarchy (see Section 6)

### Step 4: Configure Loss Function

1. **Enable Focal Loss:** (critical for rare labels)
2. Set focal gamma (4.0 recommended for severe imbalance)
3. **Enable Weighted Loss:** (critical for rare labels)
4. These work together for Weighted Focal Loss

### Step 5: Set Training Parameters

1. Epochs: 10-15 (watch for overfitting)
2. Batch size: 12-16 (depends on GPU memory)
3. Learning rate: 2e-5 (standard for RoBERTa)
4. Warmup steps: 500
5. Weight decay: 0.01-0.02

### Step 6: Configure Early Stopping

1. Enable early stopping:
2. Patience: 3 epochs
3. Min improvement: 0.001

### Step 7: Set Data Split

1. Test size: 0.2 (20%)
2. Validation size: 0.1 (10%)
3. Random seed: 42 (for reproducibility)

### Step 8: Start Training



1. Enter experiment name
2. Review summary
3. Click **Start Training**
4. Monitor real-time progress on **Monitor** tab

**Training typically takes:**

- 5,000 passages: ~30-45 minutes
- 10,000 passages: ~60-90 minutes
- GPU: 10× faster than CPU

### 3.5 Step-by-Step: Evaluating Models

#### Step 1: Load Models

1. Navigate to **Model Library** tab
2. Click **Load Model**
3. Select trained model from dropdown
4. Enter custom name (optional)
5. Click **Load Model**

#### Step 2: Run Evaluation

1. Navigate to **Evaluate** tab
2. Select model to evaluate
3. Set number of passages (100-500 recommended)
4. Enable optimal thresholds (recommended)
5. Click **Evaluate Model**

#### Step 3: Compare Models

1. Navigate to **Compare** tab
2. Select two models
3. Choose comparison type:
  - **Fair Comparison:** Sublabels only (for hierarchical vs flat)
  - **Prediction Samples:** Show actual predictions
4. Click Compare Models

## 4. Hierarchical Model Discussion

### 4.1 Previous Hierarchical Attempts

Earlier implementations of hierarchical classification (Main Categories → Sublabels) did not improve performance:

**Issues Identified:**

1. Main predictions didn't inform sublabel predictions
2. **Pipeline mismatch:** Output format incompatible with evaluation code
3. **Dimension errors:** Feature concatenation bugs in forward pass

#### **4. Overfitting:** Model memorized RA-specific patterns

### **4.2 Current Status and Next Steps**

Hierarchical experiments:

- Gated hierarchy (zero sublabels when main category confidence < threshold)
- Non-gated hierarchy (no gate thresholds)
- Teacher forcing during training (use ground truth main labels)

After multiple attempts to match the flat models best performance, we could not get a F1 micro above 0.67 even with the same dataset and configurations as the best performing flat model.

### **4.3 To Run Hierarchical Test**

Using the Golden Dataset App:

1. Load tiered dataset - the tiered set used in the best model is:  
tiered\_scored\_embedded\_cleaned\_raw\_\_Altogether\_Dataset\_RACoded\_Combined\_20251014\_091039
2. Enable "Use hierarchical structure"
3. Configure hierarchy using template or builder
4. Enable gating and set threshold
5. Use same loss configuration (Weighted Focal,  $\gamma=4.0$ )
6. Train with same parameters as flat model
7. Compare sublabel-only F1 scores

## **5. Reproducing the Best Model**

Using the Golden Dataset App:

1. Load main dataset:  
scored\_embedded\_cleaned\_raw\_\_Altogether\_Dataset\_RACoded\_Combined\_20251014\_091039
2. Navigate to Models -> Train New Model
3. Browse Tiered Data - Select:  
tiered\_scored\_embedded\_cleaned\_raw\_\_Altogether\_Dataset\_RACoded\_Combined\_20251014\_091039
4. Select: "Tier 1 + Tier 2 Combined" (5,000 passages)
5. Apply configuration from Section 1.3
6. Train for 13 epochs
7. Expected F1:  $0.72 \pm 0.01$

## **6. File Locations**

## GOLDEN\_DATASET/

- app\_golden\_dataset.py           *# Main application*
- requirements.txt           *# Python dependencies*
- .env           *# API keys (create this)*
- data/
  - cache/           *# Embedding and score caches*
    - raw\_\*\_embeddings.json   *# Stable ID → Pinecone ID mapping*
  - objects/           *# Saved data objects*
    - raw/           *# RAW stage*
    - cleaned/           *# CLEANED stage*
    - embedded/           *# EMBEDDED stage*
    - scored/           *# SCORED stage*
    - tiered/           *# TIERED stage*
- models/
- core/           *# Core functionality*
  - data\_cache.py           *# Persistent caching*
  - data\_objects.py           *# Data pipeline management*
  - data\_preparation.py           *# Cleaning and tiering*
  - quality\_scoring.py           *# Reranker system*
  - model\_training.py           *# Training engine*
  - model\_inference.py           *# Model loading/prediction*
- components/           *# UI components*
  - chat\_assistant.py           *# Chatbot*
  - data\_loader.py           *# Stable ID generation*
  - model\_manager.py           *# Multi-model management*
  - training\_monitor.py           *# Real-time training monitor*
  - ui\_helpers.py           *# Shared UI utilities*
- page\_views/           *# Streamlit pages*
  - data\_page.py           *# Data pipeline page*
  - models\_page.py           *# Training/evaluation page*
  - analysis\_page.py           *# Analyze and understand the data*
  - discover\_page.py           *# Semantic search page*

## Appendix A: Label-Specific Reranker Queries

The reranker system uses semantic queries to evaluate passage-label relevance:

```
python
LABEL_QUERIES = {
    # EVENT subcategories
    'Illness': 'Disease, sickness, illness, or mental/physical health problems',
    'Accident': 'Physical accidents, injuries, or harm not caused by illness',
    'Other': 'Other forms of misfortune, bad luck, or negative events',

    # CAUSE subcategories
    'Just_Happens': 'Events happening by chance, coincidence, or without specific cause',
    'Material_Physical': 'Physical, tangible, or natural causes for misfortune',
    'Spirits_Gods': 'Spirits, gods, deities, orixás, or supernatural entities causing problems',
    'Witchcraft_Sorcery': 'Witchcraft, sorcery, curses, or mystical malicious actions',
    'Rule_Violation_Taboo': 'Breaking rules, taboos, sins, or cultural prohibitions',

    # ACTION subcategories
    'Physical_Material': 'Physical remedies, medicine, washing wounds, protective objects,
totems',
    'Technical_Specialist': 'Medical doctors, technical experts, or specialists with practical
knowledge',
    'Divination': 'Divination, fortune telling, or procedures to reveal hidden information',
    'Shaman_Medium_Healer': 'Shamans, mediums, spirit healers, or people who interact with
spirits',
    'Priest_High_Religion': 'Priests, ordained religious authorities, or organized religious figures',
}
```

These queries are passed to VoyageAI rerank-2.5, which scores how well each passage's text matches the query semantics.

## Appendix B: Training Configuration (JSON)

Complete configuration for reproducing the best model:

```
json
{
  "base_model": "roberta-base",
  "use_hierarchy": false,
  "hidden_size": 768,
  "hierarchical_hidden_size": 768,
  "num_hidden_layers": 3,
  "dropout": 0.15,
  "attention_dropout": 0.1,
  "use_weighted_loss": true,
  "use_focal_loss": true,
  "focal_gamma": 4.0,
  "num_epochs": 13,
  "batch_size": 12,
  "learning_rate": 2e-05,
  "warmup_steps": 500,
  "weight_decay": 0.02,
  "max_length": 512,
  "test_size": 0.2,
  "validation_size": 0.1,
  "random_seed": 42
}
```